

Circle Click NeMo RL Experiment Report

Executive Summary

We ran an automated NeMo RL experiment campaign on the NeMo Gym `circle_click` task, starting from a small VLM recipe and iterating through runtime fixes, data fixes, reward shaping, batching, decoding, and targeted continuation strategies.

The best model found was `Qwen/Qwen3-VL-2B-Instruct` trained with LoRA, dense coordinate reward, forced `click` tool calls, clean 1000x1000 Circle Click data, spatial edge hints, and large GRPO batches.

Best validation result:

Rank	Attempt	Checkpoint	Accuracy	Dense reward	W&B run	Notes
1	attempt46	step90	509/512 = 99.41%	0.995575	<code>guwfx0dm</code>	Highest dense reward, tied best hit accuracy
1	attempt45	step80	509/512 = 99.41%	0.995548	<code>d8czk421</code>	First new best over 508/512
2	attempt44	step70	508/512 = 99.22%	0.994666	<code>gngwmiva</code>	Spatial hints improved dense reward
2	attempts32/34/36	step30/40/50/60	508/512 = 99.22%	up to 0.993781	<code>tskq2r7v</code> , <code>htnk8z4d</code> , <code>701c3pal</code>	Clean temp0.05 recipe saturation

The final remaining errors were not tool-call or formatting failures. They were stable visual/coordinate prior failures:

Validation index	Target	Target center/ radius	Typical model click	Failure mode
126	purple	<code>(132, 772)</code> , <code>r=110</code>	around <code>(444-464, 867-875)</code>	purple prior in lower-middle/right

Validation index	Target	Target center/ radius	Typical model click	Failure mode
246	purple	(591, 467), r=138	around (545-575, 784-788)	purple prior in lower region
296	yellow	(499, 190), r=92	around (494, 494)	image-center prior

Important lessons:

- The original 0% runs were mostly not model-size problems. They were tool-choice, coordinate-frame, reward-sparsity, and data-labeling problems.
- Dense reward was the key algorithmic breakthrough in this campaign. It converted Circle Click from a sparse pass/fail RL problem into a continuous localization problem, giving GRPO a useful ranking signal for wrong clicks. Combined with the native 1000px coordinate frame, it took validation from near 0% to 503/512.
- Clean labels mattered. The early 1000px validation split had target-absent rows, which capped accuracy and confused debugging.
- 32 prompts x 8 generations with microbatch 1 was the best basic training signal. Increasing to 64 x 8 did not improve hit accuracy, but it slightly improved dense reward when resumed from the best step80 checkpoint.
- Prompt hints helped only when they were spatial and conservative. Color-disambiguation and heavy hard-case training regressed clean validation.
- For this task, Qwen3-VL-2B beat Qwen2.5-VL-3B by a wide margin in the time budget tested.

Final Recommended Recipe

Use the Qwen3-VL-2B LoRA recipe with:

- Model: Qwen/Qwen3-VL-2B-Instruct
- Data: tutorial/data/circle_click_1000_clean_forced_tool_spatial_hint
- Training batch: num_prompts_per_step: 32 , num_generations_per_prompt: 8 ,
train_global_batch_size: 256
- Microbatch: train_micro_batch_size: 1
- Generation: temperature: 0.05 , top_p: 1.0
- Reward: dense Circle Click reward
- Checkpoint resume: checkpointing.load_optimizer: false
- Validation sample printing: logger.num_val_samples_to_print: 8
- Best continuation: start from spatial step70 and keep step80 from attempt45, or use step90 from attempt46 for highest dense score.

Relevant config files:

- `tutorial/`
`circle_click_qwen3_vl_2b_1gpu_lora_dense_reward_large_batch_1000px_clean_spatial_hint_temp005_step70_c`
- `tutorial/`
`circle_click_qwen3_vl_2b_1gpu_lora_dense_reward_large_batch_1000px_clean_spatial_hint_temp005_step80_g`

Campaign Setup

The campaign followed the local `auto_research` workflow:

- One branch per hypothesis under `autoresearch/2026-04-24-circle-click/...`
- One or more commits per experiment branch
- Untracked experiment ledger at `/ephemeral/circle-click/experiments.tsv`
- W&B logging enabled for successful and failed runs
- Runtime caches and temporary Ray directories placed under `/ephemeral` where practical

Standard environment used for runs:

```
export UV_CACHE_DIR=/ephemeral/uv-cache
export UV_PROJECT_ENVIRONMENT=/ephemeral/nemo-rl-circle-click-venv
export HF_HOME=/ephemeral/hf
export HF_DATASETS_CACHE=/ephemeral/hf/datasets
export TMPDIR=/ephemeral/tmp
export RAY_TMPDIR=/ephemeral/ray
export XDG_CACHE_HOME=/ephemeral/cache
export VLLM_CACHE_ROOT=/ephemeral/vllm-cache
export TORCHINDUCTOR_CACHE_DIR=/ephemeral/torch-cache
export CUDA_HOME=/usr/local/cuda
export TORCH_CUDA_ARCH_LIST=8.9
export MAX_JOBS=4
export CMAKE_BUILD_PARALLEL_LEVEL=4
```

W&B project:

```
circle-click-nemorl-nemogym
```

Code and Infrastructure Changes Made

Several changes were needed before model quality could be evaluated honestly.

Area	Change	Why it mattered
Data preparation	Added forced <code>tool_choice</code> and <code>agent_ref</code> to Circle Click data	Prevented free-form answers and missing agent routing

Area	Change	Why it mattered
NeMo Gym reward	Added dense reward, target metadata, center distance, and binary reward	Sparse hit-only reward gave too little signal
Validation debug	Logged <code>nemo_gym_debug</code> , printed validation samples, and wrote <code>val_data_step*.jsonl</code>	Made it clear whether failures were tool format, OOB, or visual misses
Validation metric	Overrode validation accuracy with hit rate when debug exists	Avoided ambiguity between dense reward and hit accuracy
Checkpointing	Added <code>checkpointing.load_optimizer: false</code>	LoRA optimizer restore failed on continuation checkpoints
Ray packaging	Excluded large directories from Ray package	Reduced package size to about 133 MiB and sped startup
Circle Click generator	Fixed target-label/data cleanliness issues	Removed target-absent validation rows

Dense Reward: The Algorithmic Breakthrough

Dense reward was not present in the original upstream Circle Click task. It was an experiment-local reward-shaping implementation added to the NeMo Gym `circle_click` resource server. I would describe it as the decisive algorithmic discovery of this campaign: not a claim of broad research novelty, but the new idea that made this particular RL setup learnable.

Original Reward

The original task was sparse and binary:

```
reward = 1.0 if click lands inside the requested circle
reward = 0.0 otherwise
```

That is a harsh signal for coordinate learning. A click 1 pixel outside the target and a click 800 pixels away both receive `0.0`. For GRPO, that means most early rollouts are indistinguishable failures, especially when the model is still learning the 1000x1000 coordinate frame.

New Reward

The dense reward keeps exact hits at `1.0`, but gives misses a smooth score based on distance to the target center:

```

if hit:
    reward = 1.0
else:
    reward = exp(-center_distance / (target_radius * dense_reward_temperature))

```

The implementation lives in `3rdparty/Gym-workspace/Gym/resources_servers/circle_click/app.py` :

```

def _shaped_reward(self, distance: float, radius: float, hit: bool) -> float:
    if hit:
        return 1.0
    scale = max(radius * self.config.dense_reward_temperature, 1.0)
    return math.exp(-distance / scale)

```

The campaign used:

```

dense_reward: true
dense_reward_temperature: 2.0

```

Why It Worked

The shaped reward gives GRPO a gradient-like preference ordering without changing the environment action space:

Click quality	Binary reward	Dense reward behavior
Inside the target circle	1.0	1.0
Just outside the circle	0.0	high partial credit
Same quadrant but far from center	0.0	medium/low partial credit
Wrong side of the image	0.0	near-zero credit
Bad tool call/no coordinates	0.0	0.0

This turned "all misses are equal" into "closer misses are better." That was exactly the missing signal for a VLM that could see the objects and call the tool, but had not yet calibrated pixel coordinates.

Evidence from the Campaign

The dense reward alone did not instantly solve the task. Attempt23 showed nonzero reward but poor validation because the coordinate frame and data plumbing were still wrong. The breakthrough came when dense reward was combined with:

- forced `click` tool choice,
- native 1000x1000 coordinates,
- valid `agent_ref` routing,

- and larger 32 x 8 GRPO rollout batches.

The before/after pattern was stark:

Stage	Reward setup	Coordinate/data setup	Validation
attempts20-22	sparse/binary or incomplete dense path	forced tool, but bad coordinates/OOB	0.0% to 0.2%
attempt23	dense reward	still not enough signal by itself	about 0.4%
attempt24	dense reward + 32 x 8	still high OOB	about 0.2%
attempt26	dense reward + 32 x 8	native 1000px + fixed agent ref	503/512 = 98.24%

So the key discovery was not just "use dense reward" in isolation. It was that Circle Click should be treated as a continuous visual localization problem during RL training, while preserving binary hit accuracy as the evaluation metric.

Metric Separation

To avoid fooling ourselves, validation reported both:

- accuracy : true hit rate from the hit boolean
- dense_reward : average shaped reward

This mattered because dense reward can improve even when hit accuracy ties. For example, attempt46 tied the best hit accuracy at 509/512 , but improved dense reward from 0.995548 to 0.995575 , meaning the successful and near-miss clicks were slightly closer on average.

Attempt Timeline

Phase 1: Qwen2.5-VL-3B Baseline and Runtime Bring-Up, Attempts 1-13

Initial work focused on Qwen/Qwen2.5-VL-3B-Instruct , since it was the small VLM recipe requested at the start.

Key outcomes:

- attempts1-9 mostly exposed environment and Ray worker build issues:
- missing InfiniBand headers
- inherited CLEVR config keys
- missing data.use_multiple_dataloader

- editable Gym cache packaging issues
- transformer-engine/nv-grouped-gemm build issues
- need for `TORCH_CUDA_ARCH_LIST=8.9`
- attempt10 reached initial validation but had 0% accuracy and OOMed during the first optimizer step.
- attempts11-13 moved to a LoRA-only low-memory Qwen2.5 recipe. Training became stable through step10 at about 28 GiB peak, but accuracy and train reward stayed at 0%.

Conclusion:

Qwen2.5-VL-3B was usable after memory fixes, but it did not learn the task under the early sparse/tool setup. We pivoted to Qwen3-VL-2B because it had better tool-calling compatibility with the available vLLM parser.

Phase 2: Qwen3-VL-2B Tool Calling, Attempts 14-22

The first Qwen3-VL-2B runs still scored 0% until tool calling was forced correctly.

Key attempts:

Attempt	Result	Interpretation
14	Crash	vLLM served Qwen3-VL, but policy loading used the wrong AutoModel path
15	0%	Model loaded, but train and validation reward stayed 0
16-18	Crashes	Config inheritance and stale checkpoint issues
19	Failed after promising behavior	Forced tool choice hit about 91% validation before service failure
20	0.2%	Named <code>click</code> tool stabilized, but sparse reward still weak
22	0%	Tool calls valid, but coordinates were badly out of bounds

Conclusion:

The model could produce valid tool calls, but sparse reward was not enough to correct coordinate behavior. Debug prints showed valid calls with wrong/OOB coordinates, so the next step was reward shaping.

Phase 3: Dense Reward, Large Batch, and Native 1000px Coordinates, Attempts 23-27

This phase produced the first high-accuracy result.

Attempt	Hypothesis	Result
23	Dense reward gives coordinate signal	Nonzero reward, but only about 0.4% validation
24	Larger batch 32 x 8 improves GRPO signal	Stable but still poor, with high OOB rate
25	Native 1000px coordinate frame	Crashed due to missing agent_ref
26	Native 1000px plus fixed agent ref	503/512 = 98.24%
27	Clean labels	500/512 = 97.66%, step20 regressed

The jump at attempt26 was the biggest single improvement in the campaign. Dense reward was the central algorithmic mechanism: it supplied partial credit for coordinate localization, while native 1000px coordinates made that signal correspond to the actual image. The task became learnable once:

- the coordinate frame matched the image size,
- the tool was forced,
- dense reward provided distance signal,
- and the model was trained with 32 x 8 rollouts.

Phase 4: Temperature and Clean Continuations, Attempts 28-36

This phase improved from 503/512 to 508/512.

Attempt	Result	Notes
28	506/512	temperature: 0.2 reduced jitter
29	506/512	Purple oversampling did not transfer
30	507/512	temperature: 0.05 became new best
31	Failed	Optimizer resume issue
32	508/512	Weight-only resume fixed continuation
33	Failed	Optimizer resume still broken without weight-only
34	508/512	Tied best at steps40 and 50
35	506/512	Purple-only continuation regressed
36	508/512	Step60 had best dense reward so far, step70 regressed

Key finding:

Clean continuation at temperature: 0.05 worked better than targeted purple training. The model saturated at 508/512, with remaining misses mostly purple.

Phase 5: Hard-Case, LR, Prompt, and Alternative Model Probes, Attempts 37-43

This phase tried to break the 508/512 ceiling.

Attempt	Hypothesis	Result
37	25% hard-purple mix	506/512, regressed
38	5% hard-purple mix	507/512, regressed
39	Lower LR continuation	507/512
40	Lower temperature 0.01 continuation	505/512
41	Purple disambiguation prompt	503/512
42	Qwen2.5-VL-3B clean large batch	361/512 at step10
43	GBS512 from step60	507/512

Conclusion:

Targeted hard data and color-oriented prompt hints overcorrected or disrupted clean validation. Qwen2.5-VL-3B was far behind Qwen3-VL-2B on this setup. Larger GBS from step60 did not help before the spatial hint was introduced.

Phase 6: Spatial Hints and New Best, Attempts 44-46

This phase introduced a conservative spatial instruction:

The target circle may be close to an image edge or corner. Click the actual visible center of the requested colored circle, and do not default to the image center or a common prior location.

For purple targets, the user text also included a short edge-location reminder.

Attempt	Result	Notes
44	508/512, dense 0.994666	Tied hit best, new dense best
45	509/512 at step80, dense 0.995548	First 99.41% run
46	509/512 at step90, dense 0.995575	GBS512 tied best and improved dense slightly

This was the successful final move. Spatial hints reduced the purple-heavy miss set from four misses to three misses at the best checkpoint.

Phase 7: Residual Failure Probes, Attempts 47-49

After reaching 509/512, we tried to convert the last three failures.

Attempt	Hypothesis	Result
47	Small targeted anti-prior synthetic mix	507/512, added cyan/orange misses
48	Near-greedy validation	508/512, vLLM clamped temp to 0.01
49	Warm temp0.1 validation	508/512

Conclusion:

The residual errors were not simple decoding-temperature issues. Targeted synthetic anti-prior data was too disruptive when mixed into continued RL training.

Detailed Best Run Analysis

attempt45

Branch:

autoresearch/2026-04-24-circle-click/qwen3-spatial-hint-continue90

Commit:

3ed6efa

W&B:

d8czk421

Best validation at step80:

```
hit_acc=0.994141
hits=509/512
dense_mean=0.995548
median_dist=17.804
p90_dist=37.560
bad_tool=0
oob=0
miss_colors={'purple': 2, 'yellow': 1}
```

Step90 regressed to 508/512, so step80 should be kept.

attempt46

Branch:

```
autoresearch/2026-04-24-circle-click/qwen3-step80-gbs512-continue90
```

Commit:

```
6c71b13
```

W&B:

```
guwfx0dm
```

Validation at step90:

```
hit_acc=0.994141
hits=509/512
dense_mean=0.995575
median_dist=17.117
p90_dist=37.980
bad_tool=0
oob=0
miss_colors={'purple': 2, 'yellow': 1}
```

This did not improve hit accuracy over attempt45 but did slightly improve dense reward and median distance.

Why the Final 3 Misses Persisted

The last misses were spatial priors rather than API failures:

- The model called `click` correctly.
- The arguments were valid integer coordinates.
- The coordinates were inside the 1000x1000 image.
- The clicks clustered around plausible learned positions instead of the visible target centers.

The persistent samples:

```
idx126 purple target (132,772), r=110 -> click around (444-464,867-875)
idx246 purple target (591,467), r=138 -> click around (545-575,784-788)
idx296 yellow target (499,190), r=92 -> click around (494,494)
```

This suggests the final errors are from vision localization or learned coordinate priors, not from tool syntax, color vocabulary, or reward plumbing.

What Worked

The winning stack was:

- 1. Qwen3-VL-2B instead of Qwen2.5-VL-3B for this tool-calling path.
- 2. Forced named `click` tool choice.
- 3. Native 1000px coordinate frame.
- 4. Clean validation labels with no target-absent rows.
- 5. Dense reward based on distance to target center, which made wrong clicks rankable by quality instead of all receiving the same zero.
- 6. Larger rollout signal: `32 prompts x 8 generations`.
- 7. `temperature: 0.05` for training and validation.
- 8. Weight-only continuation to avoid LoRA optimizer restore failures.
- 9. Spatial edge/corner hinting instead of color-disambiguation hinting.
- 10. Validation sample printing plus JSONL debug records.

What Did Not Work

Strategy	Outcome
Sparse reward only	Stayed near 0% or had no useful coordinate signal
Tiny batch size	Too noisy for coordinate learning
Qwen2.5-VL-3B in this setup	Stable but far below Qwen3-VL-2B
Purple-only continuation	Regressed clean validation
Hard-purple mixtures	Overfit/regressed clean validation
Color-disambiguation prompt	Made validation worse
Very low temperature continuation	Regressed
Near-greedy validation	Regressed to 508/512
Warm temp0.1 validation	Regressed to 508/512
Targeted anti-prior synthetic continuation	Regressed and added new misses

Reproducibility Notes

Use `/ephemeral` for the large and volatile directories:

```
export UV_CACHE_DIR=/ephemeral/uv-cache
export UV_PROJECT_ENVIRONMENT=/ephemeral/nemo-rl-circle-click-venv
export HF_HOME=/ephemeral/hf
export HF_DATASETS_CACHE=/ephemeral/hf/datasets
export TMPDIR=/ephemeral/tmp
export RAY_TMPDIR=/ephemeral/ray
export XDG_CACHE_HOME=/ephemeral/cache
export VLLM_CACHE_ROOT=/ephemeral/vllm-cache
export TORCHINDUCTOR_CACHE_DIR=/ephemeral/torch-cache
```

Before each run:

```
UV_PROJECT_ENVIRONMENT=/ephemeral/nemo-rl-circle-click-venv \
UV_CACHE_DIR=/ephemeral/uv-cache \
uv run ray stop --force >/dev/null 2>&1 || true

rm -rf /ephemeral/ray/* /ephemeral/tmp/*
```

Main launch command:

```
uv run --extra nemo_gym --extra vllm \
  examples/nemo_gym/run_grpo_nemo_gym.py \
  --config <config.yaml>
```

Validation parser used during the campaign:

```
UV_PROJECT_ENVIRONMENT=/ephemeral/nemo-rl-circle-click-venv \
UV_CACHE_DIR=/ephemeral/uv-cache \
uv run python - <<'PY'
import json, glob, statistics

for path in sorted(glob.glob("logs/<logdir>/exp_*/val_data_step*.jsonl")):
    rewards = []
    hits = 0
    n = 0
    miss_colors = {}
    misses = []
    bad_tool = 0
    oob = 0
    dists = []

    for i, line in enumerate(open(path)):
        row = json.loads(line)
        reward = row.get("rewards")
        if isinstance(reward, list):
            reward = reward[0]
        rewards.append(float(reward))
        n += 1

        debug = row.get("nemo_gym_debug")
        if isinstance(debug, list):
            debug = debug[0]
        debug = json.loads(debug) if isinstance(debug, str) else (debug or {})
```

```

hit = bool(debug.get("hit"))
hits += hit
x = debug.get("clicked_x")
y = debug.get("clicked_y")
if x is None or y is None:
    bad_tool += 1
elif not (0 <= float(x) <= 1000 and 0 <= float(y) <= 1000):
    oob += 1
if debug.get("center_distance") is not None:
    dists.append(float(debug["center_distance"]))
if not hit:
    color = debug.get("target_color")
    miss_colors[color] = miss_colors.get(color, 0) + 1
    if len(misses) < 12:
        misses.append((i, color, debug.get("target_circle"), x, y,
debug.get("center_distance")))

step = path.rsplit("step", 1)[1].split(".")[0]
print(
    f"step={step} hit_acc={hits/n:.6f} hits={hits}/{n} "
    f"dense_mean={sum(rewards)/n:.6f} median_dist={statistics.median(dists):.3f} "
    f"p90_dist={statistics.quantiles(dists, n=10)[8]:.3f} "
    f"bad_tool={bad_tool} oob={oob} miss_colors={miss_colors}"
)
print(" misses", misses)
PY

```

Open Items

- Persisting a known-good container image was requested but has not been completed in this report. The runtime is reproducible from the current workspace and `/ephemeral` cache setup, but no Docker image tag has been recorded here yet.
- The tutorial file should be updated to point users to the Qwen3-VL-2B final recipe as the highest-accuracy path. The older Qwen2.5 tutorial remains useful as a small-model starting point, but it was not the winning recipe.
- A future run could try a different VLM family or a supervised coordinate warm-start on the three residual failure patterns, then resume GRPO. The RL-only targeted hard mixes were too disruptive.

Bottom Line

The campaign reached high accuracy on Circle Click:

```

Best hit accuracy: 509/512 = 99.41%
Best dense reward: 0.995575
Best model: Qwen/Qwen3-VL-2B-Instruct with LoRA

```

Best recipe family: dense reward + forced tool + clean 1000px data + spatial hint + 32x8 or 64x8 GRPO continuation

At this point, the remaining gap appears to be a small number of stable visual localization priors rather than any NeMo RL, NeMo Gym, tool-call, or coordinate-format issue.